

Halaqohv2 Codebase Description

Halaqohv2 — Codebase Description

Overview

Halaqohv2 is a Node.js/Express web application for managing and clustering "santri" (students) into halaqoh groups using a K-Means implementation. The app provides web views (EJS) for administrators and API endpoints for running clustering and retrieving results. It stores basic data in a relational database (MySQL) via `config/database.js`.

High-level architecture

- Runtime: Node.js with Express server (entry: `index.js`).
- Views: EJS templates in `views/` organized by feature (dashboard, clustering, master data, auth, layouts, partials).
- Static assets: `public/` containing CSS and JS used by client pages.
- Controllers: two sets:
 - `controllers/api/` - JSON APIs for operations like clustering, managing users, santri, halaqoh, etc.
 - `controllers/web/` - controllers that render EJS pages and handle web routes.
- Routes: `routes/` organizes API and web routes; top-level route files are `routes/web.js`, `routes/api.js`, `routes/auth.js` and sub-route files under `routes/api/`.
- Services: business logic and algorithms in `services/` (notably `clustering.js` for K-Means).
- Middleware: `middleware/` holds authentication and audit middlewares.
- Config: `config/database.js` sets up DB connection (MySQL pool) used across controllers.

Key modules and responsibilities

1. `services/clustering.js`

- Implements a K-Means algorithm using Manhattan distance.
- Functions include: `manhattanDistance`, `randomCentroids`, `assignClusters`, `updateCentroids`, `silhouetteScore`, and `kmeans`.
 - `kmeans` returns: `{ clusters, centroids, silhouette, assignments, points }` where `clusters` maps cluster index to member IDs, `assignments` is an array of cluster indexes per input data index, and `points` is simplified array with `id`, `features`, and assigned `cluster`.

2. `controllers/api/clusteringController.js`

- Fetches `santri` records (fields: `juz`, `tajwid`, `kelancaran`, `makhraj`) and maps them to feature vectors.
- Calls `services.clustering.kmeans` and computes `clusterStats` including labeled categories (Tinggi/Menengah/Rendah) based on `avgScore`.
- Prepares `points` array used by frontend; stores clustering result in `sessionStorage` from frontend.

3. Views and Frontend

- `views/dashboard/clustering.ejs`: UI to run clustering (POST to `/api/clustering/process`) and stores result to `sessionStorage` for the result page.
- `views/dashboard/result.ejs`: Renders charts (Chart.js) and a table of labeled santri; contains client-side sorting utilities and CSV/PDF download helpers.

- ``public/js/`` contains additional client-side helpers, e.g., `clustering-frontend.js` used in dashboard pages.

4. Routes

- ``routes/api.js`` and ``routes/web.js`` wire controllers to URL paths for API and web respectively.

Data flow

- Admin triggers clustering from ``/clustering`` page.
- Client sends `POST `/api/clustering/process`` with parameters (`k`, `maxIter`).
- Server reads ``santri`` from database, runs ``kmeans``, computes silhouettes and labels, and responds with JSON containing clusters, centroids, silhouette, assignments, points, clusterSizes, and clusterStats.
- Client stores the JSON into ``sessionStorage`` under ``clusteringResult`` and then navigates to ``/result`` where ``views/dashboard/result.ejs`` reads from ``sessionStorage``, renders charts and the table, and allows CSV/PDF export.

Database

- Database access is centralized in ``config/database.js`` using a MySQL connection pool.
- There is a ``database.sql`` file in the repo containing DB schema or seed data (open for review if needed).

Developer notes and potential improvements

- Sorting: The frontend sorts ``points`` by ``label``, ``name``, or ``cluster`` client-side. Recent fixes improved cluster sorting stability and NaN handling. Consider moving sorting to server-side when results are large.
- PDF generation: The UI offers client-side PDF generation using ``jspdf`` and ``jspdf-autotable``. For large exports or better styling, consider server-side PDF generation.
- K-Means: The current implementation uses Manhattan distance and random centroid initialization. For reproducible results, consider allowing seeded initialization (`kmeans++` or deterministic sampling).
- Tests: Add unit tests for ``services/clustering.js`` (edge cases: empty dataset, clusters with zero members, silhouette computation correctness).
- UX: Consider pagination or virtual scrolling in the ``result`` table for many santri.

How to run locally (developer)

1. Install dependencies:

```
```bash
npm install
```
```

2. Configure the database in ``config/database.js`` (set host/user/password/database).

3. Start the server:

```
```bash
npm start
or
node index.js
```
```

4. Open the app in a browser and navigate to `/clustering` to run clustering and `/result` to view results.

Files of interest (quick map)

- `index.js` - app entry
- `config/database.js` - DB pool
- `controllers/api/clusteringController.js` - clustering API
- `services/clustering.js` - K-Means implementation
- `views/dashboard/clustering.ejs` - run UI
- `views/dashboard/result.ejs` - results UI + sorting logic
- `public/js/clustering-frontend.js` - frontend helpers

Contact & next steps

If you want a packaged PDF inside `.agent` I can generate it now using Puppeteer; I will add a small script into `.agent/` that converts this markdown to PDF and then run it to create `.agent/halaqohv2_description.pdf`. If you prefer a plain README (MD) only, I can skip PDF generation.